MABEL Dexterous Manipulation in mjlab: A GPU-Accelerated RL Environment for Tabletop Arm–Hand Control

MABEL Robotics — Reinforcement Learning Subsystem

July 3, 2026

Abstract

We build a GPU-accelerated reinforcement-learning environment for the MABEL robot on top of *mjlab* (Isaac-Lab's manager-based API powered by MuJoCo-Warp). MABEL is a dual-arm humanoid on a three-module swerve base with two 7-DOF arms and two 17-DOF ORCA hands. Following the GeoDEX/xArm+ORCA recipe, we reduce the canonical robot MJCF to a fixed-base tabletop platform and expose a staged task progression: single arm+hand, dual arm+hand, and whole-body mobile manipulation. This document covers Phase 1 — a right 7-DOF arm plus 16-DOF ORCA hand (24 actuated DOF) lifting a cube from a table — Phase 2 — both arms and hands (48 DOF) lifting a wide box that a single hand cannot manage — and Phase 3 — a whole-body (upper-body) variant that additionally actuates the coupled lift column and torso pitch (26 DOF). The reduced models are *derived* at load time from the single source of truth `mabel_full.xml`, so they never drift from the canonical robot. We report the measured reachable workspace, home-pose gravity stability (max drift 2.7°), and validate both tasks end-to-end on CPU (mujoco-warp) with dummy agents; PPO training runs on an NVIDIA GPU.

Keywords: dexterous manipulation, reinforcement learning, MuJoCo-Warp, mjlab, ORCA hand, sim-to-real, tabletop grasping.

1 Overview

mjlab [1] composes environments from modular managers (observations, actions, rewards, terminations, events, curriculum) over MuJoCo-Warp for massively parallel GPU simulation. A robot is an MJCF (geometry, joints, collision geoms, sites) plus a Python constants module that attaches position actuators, PD gains, effort limits, a home pose, and collision configuration — exactly the pattern of the reference i2rt-YAM lift-cube task. We add MABEL to this framework so that `train/play Mjlab-Lift-Cube-Mabel` works in-tree.

MABEL's canonical model `mabel_full.xml` (67 `qpos`, 66 `qvel`, 59 actuators, floating base) is the single source of truth for every derived representation in the wider repository. Rather than fork a separate manipulation MJCF, we *derive* the RL model from it programmatically (§2.1); editing the canonical MJCF flows through automatically.

2 Method

2.1 Model derivation

Given `mabel_full.xml`, `derive_model.build_manip_spec` produces a reduced `mujoco.MjSpec`:

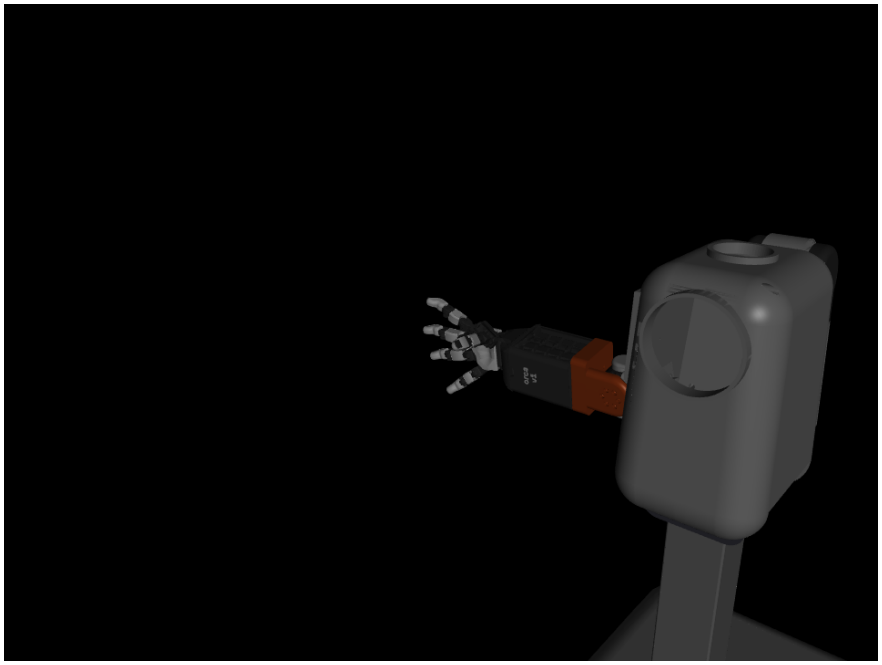


Figure 1: Derived Phase-1 model: fixed pedestal (chassis + lift + torso) carrying the right 7-DOF arm and 16-DOF ORCA hand, shown at $q = 0$.

1. Strip all `<actuator>`, `<sensor>` (a 360-ray lidar + IMU), `<tendon>` (the coupled lift), and `<equality>` (`Sync Lift`) elements. mjlab re-attaches position actuators from the constants module, so the MJCF carries none.
2. Delete the base free joint `base_root`, making the chassis a world-fixed pedestal. mjlab auto-wraps the fixed-base entity in a mocap body for per-environment placement.
3. Prune the subtrees not used for tabletop manipulation: the three swerve modules, the neck/head, the lidar, and (for the single-arm phase) the left arm. The lift and torso joints are removed so the pedestal is rigid.
4. Add a palm grasp site `right_grasp_site` at the palm-local grasp centre $(0, 0, 0.102\,\mathrm{m})$ and one site per fingertip; name the distal fingertip collision geoms `right_{thumb,index,middle,ring,pinky}_tip_collision` (the canonical model tags collision geoms by class but leaves them unnamed, and the reward/randomization terms select geoms by name).

The result (Fig. 1) has $n_q = n_v = 24$: a 7-DOF arm, a 1-DOF wrist, and a 16-DOF ORCA hand.

2.2 Actuators

Each remaining joint receives an mjlab `BuiltinPositionActuatorCfg`, using the gains authored in `mabel_full.xml`: arm joints at $k_p = 100$, $k_v = 20$ with per-tier effort limits (40 N m shoulder DM-J8009, 27 N m mid DM-J4340, 7 N m distal DM-J4310); wrist and finger joints at $k_p = 2$, effort 1 N m (the ORCA `hand_base` default). Joint armature, passive damping, and frictionloss are preserved from the canonical model. The position-control law for actuator i is

$$\tau_i = \mathrm{clip}\big(k_{p,i}(q_i^* - q_i) - k_{v,i}\dot{q}_i,\ -\bar{\tau}_i,\ \bar{\tau}_i\big), \tag{1}$$

with target $q_i^* = q_i^{\text{home}} + s_i\, a_i$ for policy action $a_i \in [-1, 1]$ and per-actuator scale $s_i = 0.25\,\bar{\tau}_i / k_{p,i}$ (Fig. 4), following the YAM convention.

2.3 Task MDP

The task reuses the shared `make_lift_cube_env_cfg` template. **Observations** (78-dim actor/critic): joint positions and velocities (relative to home), grasp-site→cube vector, cube→goal vector, and last action. **Action** (24-dim): position targets for all arm and hand joints. **Reward**: a staged reaching+lifting term, a precise bring-to-goal term, plus action-rate, joint-limit, and joint-velocity penalties. **Events**: per-environment reset of the robot and table to their env origin, home-pose reset, and startup fingertip-friction domain randomization. **Termination**: time-out and illegal hand-ground contact. The cube spawns on a 0.5 m-tall table centred at $(x, y) = (-0.30, 0.38)$ within the arm's reachable workspace, and a dynamic goal commands a lift of 10 cm to 25 cm.

3 Experiments

All numbers are measured from the compiled model / mjlab entity by the seeded harness `mabel/experiments/generate_figures.py` (seed = 0); no values are hand-authored.

Table 1: Phase-1 environment summary (measured).

Quantity	Value	Notes
Actuated DOF	24	7 arm + 1 wrist + 16 finger
Observation dim	78	actor and critic
Reach x [m]	$[-0.87, +0.65]$	grasp-site, full joint sweep
Reach y [m]	$[-0.45, +0.95]$	
Reach z [m]	$[+0.14, +1.70]$	
Home hand $[x, y, z]$ [m]	$(-0.30, 0.38, 0.64)$	above table
Home hold max drift	2.72°	1.5 s under gravity
Home hold mean drift	0.59°	
Cube rest height [m]	0.52	on table, all envs

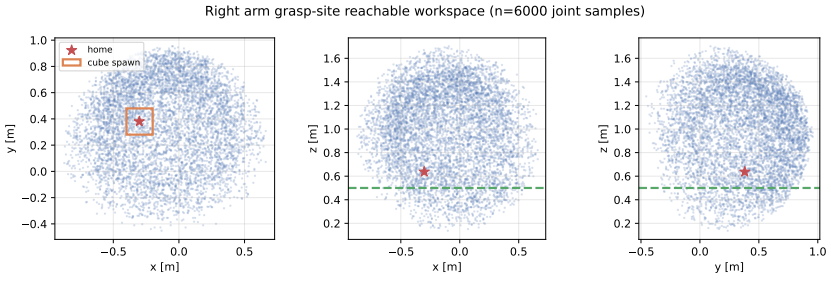


Figure 2: Right-arm grasp-site reachable workspace (6000 uniform joint samples), with the home hand ($\star$), the cube-spawn box, and the table surface overlaid. The tabletop task region lies well inside the envelope.

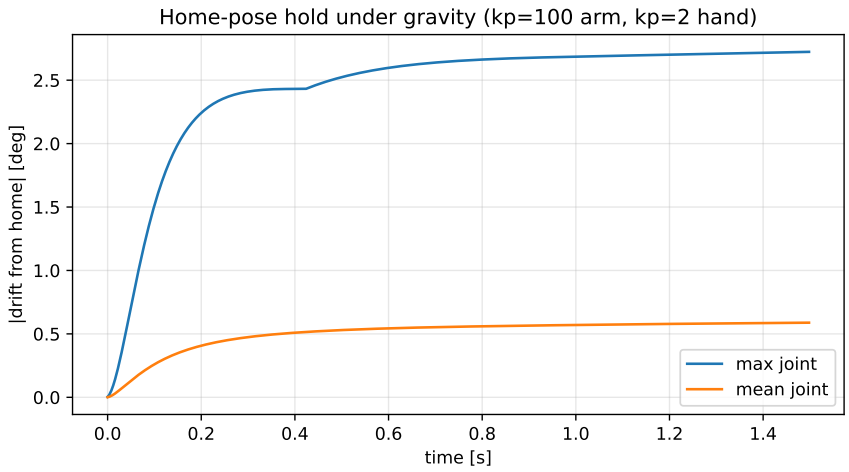


Figure 3: Home-pose hold under gravity with the position actuators commanding the home targets. The IK-designed home settles with $< 3^\circ$ peak joint drift and no divergence.

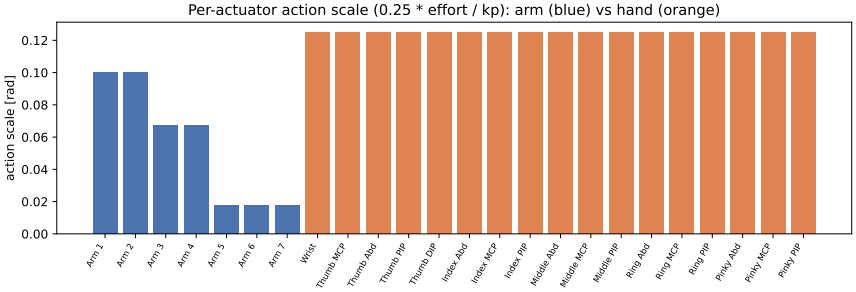


Figure 4: Per-actuator action scale $s_i = 0.25\,\bar{\tau}_i / k_{p,i}$. Arm joints (blue) span 0.018 rad to 0.1 rad; hand joints (orange) are uniform at 0.125 rad.

End-to-end validation. On macOS (mujoco-warp CPU backend, evaluation only) the environment builds, resets, and steps with zero and random agents over multiple environments with no NaNs; the cube rests on the table ($z = 0.52$) in every environment; and the eight-test suite `tests/test_mabel_manipulation.py` passes. This exercises the whole MDP stack short of the PPO update, which requires an NVIDIA GPU.

4 Phase 2: Bimanual Box Lift

The same derivation with `sides=("left","right")` yields a 48-DOF dual-arm platform (both grasp sites present). Task `Mjlab-Lift-Box-Mabel-Bimanual` places a 18 cm-wide, 0.4 kg box on a table centred on the robot's midline; symmetric IK homes bring each hand to a side of the box (Fig. 6). Because the shared reward terms assume a single end-effector, we add two-hand variants (`bimanual_mdp`): a reach reward $\propto \exp\big(-(d_L^2 + d_R^2)/\sigma^2\big)$ over both grasp sites, a hand- balance penalty $|d_L - d_R|^2$, and a paired reach observation. The env exposes a 48-dim action and a 153-dim observation, builds and steps without NaNs on CPU, and the box rests on the table ($z = 0.55$) across environments.

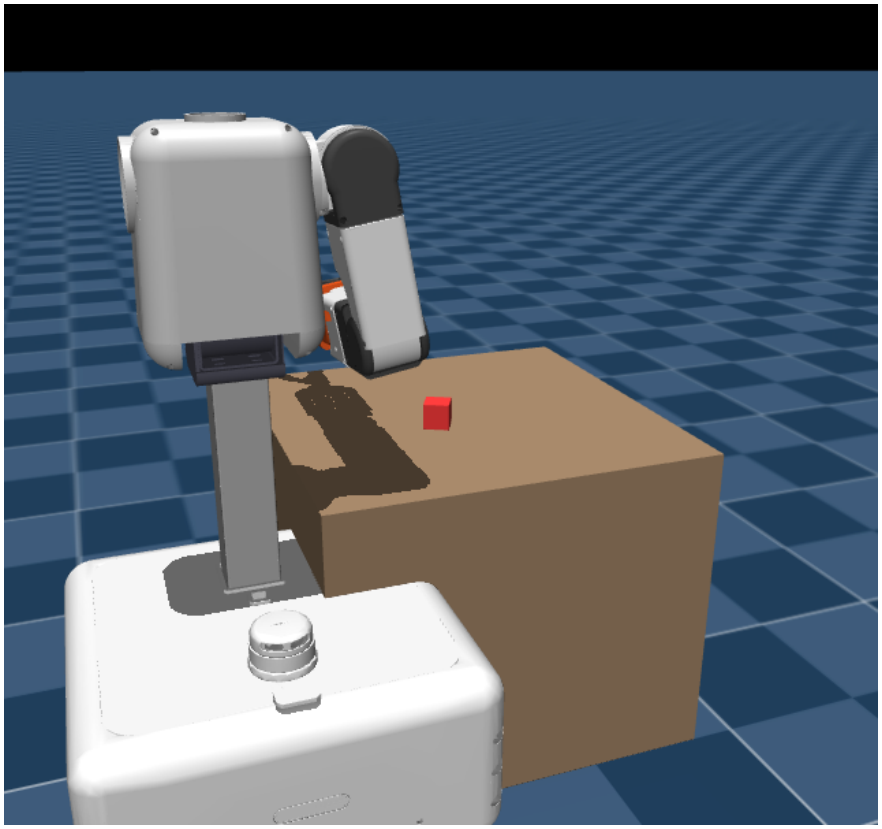


Figure 5: The assembled `Mjlab-Lift-Cube-Mabel` scene: MABEL on its swerve base with the right arm+ORCA hand at home, the red cube resting on the table.

5 Phase 3: Whole-Body (Upper-Body) Lift

Passing `keep_lift=True, keep_torso=True` to the derivation keeps the lift column and torso pitch active (27 `qpos`: the 24 arm/hand DOF plus the two lift slides and the torso). The lift is two prismatic joints coupled by the retained `Sync Lift` equality, so a single position actuator on `Lift Lower` drives both stages together — verified in MuJoCo, where a 0.28 m command moves both slides in lock-step, giving up to ∼0.5 m of coordinated vertical hand travel. Task `Mjlab-Lift-Cube-Mabel-WholeBody` (26 actions: 24 + lift + torso) uses the same Phase-1 workspace but raises the lift-goal ceiling to 45 cm above the table, so tall goals reward raising the whole body rather than over-extending the arm (Fig. 7). Because the lift/torso links are heavy ($k_p = 10^4$ / 5×10^3), the effort/k_p action scale is negligible; we use explicit scales (0.2 m, 0.2 rad per unit action) so the policy can actually move them. The heavy lift holds its home under gravity to within 9 mm, and the env builds/steps without NaNs.

6 Implementation & Usage

```
# from learning/reinforcement_learning/mjlab
uv run python -m mjlab.asset_zoo.robots.mabel.derive_model --sides right --view
uv run python mabel/experiments/generate_figures.py     # regenerate figures
uv run pytest tests/test_mabel_manipulation.py          # validate (CPU/Mac)
```

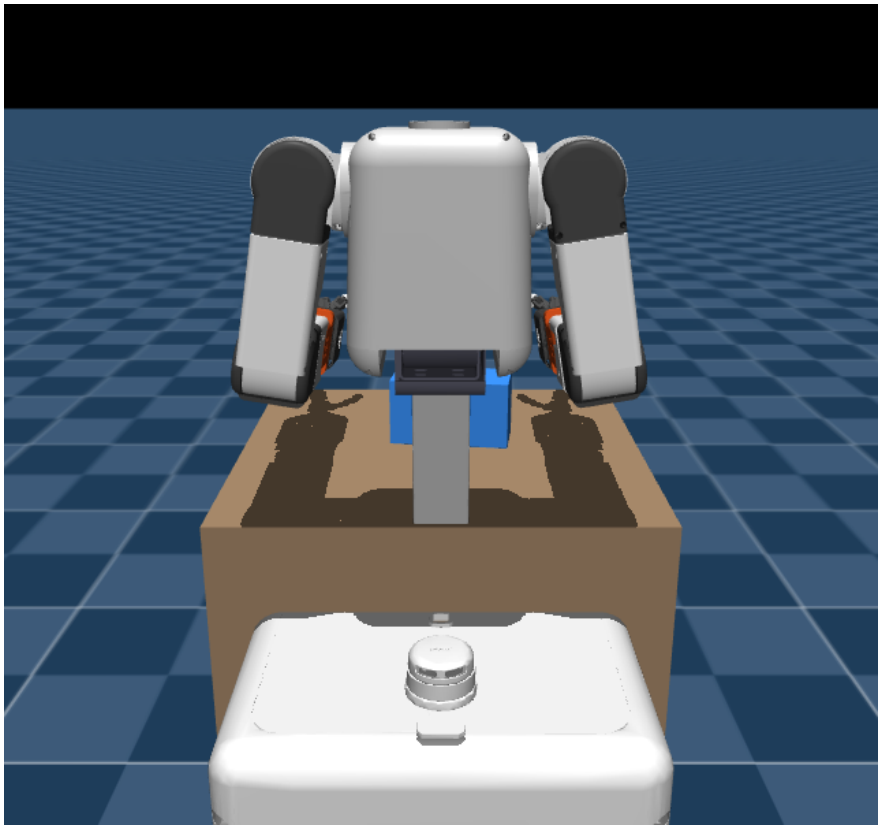


Figure 6: `Mjlab-Lift-Box-Mabel-Bimanual`: both arms + ORCA hands framing the box on the midline table, at the symmetric bimanual home.

```
uv run play Mjlab-Lift-Cube-Mabel --agent zero          # sanity (eval)
uv run train Mjlab-Lift-Cube-Mabel --env.scene.num-envs 4096         # GPU only
uv run train Mjlab-Lift-Box-Mabel-Bimanual --env.scene.num-envs 4096 # GPU only
uv run train Mjlab-Lift-Cube-Mabel-WholeBody --env.scene.num-envs 4096 # GPU only
```

Code lives in `src/mjlab/asset_zoo/robots/mabel/` (model + constants) and `src/mjlab/tasks/manipulation/config/mabel/` (task + RL config). The task self-registers via mjlab's package importer.

7 Limitations

- **Training is GPU-only.** mujoco-warp on macOS is evaluation-only; PPO must run on an NVIDIA/CUDA host.
- **Soft position gains.** $k_p = 100$ leaves $\sim 2.7°$ gravity droop at the home pose; the policy compensates, but a stiffer or gravity-compensated controller may improve sample efficiency.
- **Swerve base still fixed.** Phase 3 unfreezes the lift and torso, but the mobile swerve base remains fixed; true mobile manipulation (`keep_base=True`, floating base, wheel contact) is the next step and is expected to be substantially harder to train.
- **Primitive object + no vision.** A single cube with proprioceptive state; camera observations and the scanned-object library (`mabel_manip_table.xml`) are future work.

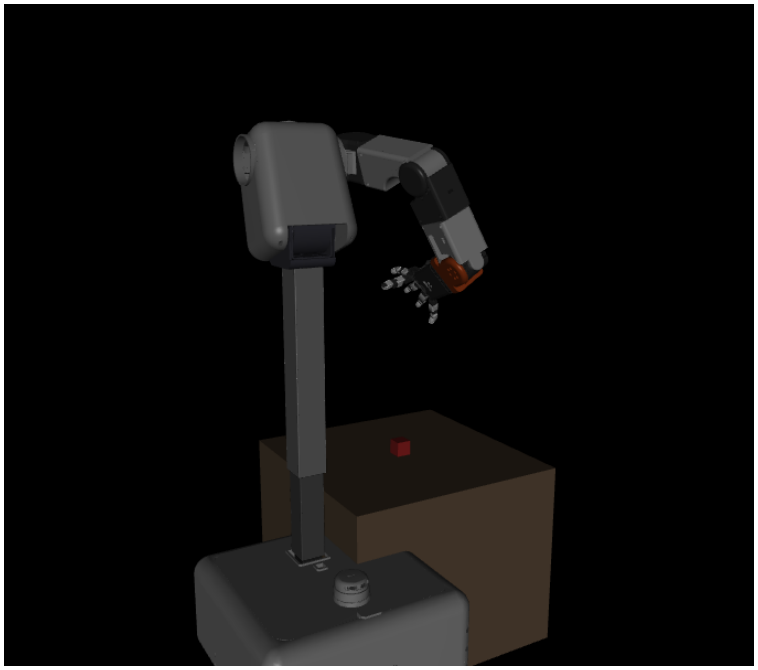


Figure 7: Whole-body pose: the lift column raised and torso pitched forward extend the arm+hand's reach toward the cube — coordination the policy learns for tall lift goals.

- **Contact fidelity.** `noslip` and elliptic-cone settings from the CPU model may need revisiting for the Warp solver at scale.

References

[1] K. Zakka, Q. Liao, B. Yi, L. Le Lay, K. Sreenath, P. Abbeel. *mjlab: A Lightweight Framework for GPU-Accelerated Robot Learning*. arXiv:2601.22074, 2026.

[2] C. ETH SRL. *ORCA: An Open-Source, Reliable, Cost-Effective, Anthropomorphic Robotic Hand*. arXiv:2504.04259, 2025.

[3] DeepMind. *MuJoCo Warp: GPU-optimized MuJoCo*. `https://github.com/google-deepmind/mujoco_warp`.